
Cómputo Nube y Tecnologías Emergentes

Instituto Nacional de Astrofísica, Óptica y Electrónica (INAOE) - Maestría en Ciencias en Tecnologías de Seguridad
Lázaro Bustio-Martínez, PhD
lbustio@inaoe.mx
Verano 2026
96 horas
- Práctica -

1. Propósito de la sesión

Esta sesión no introduce una práctica completamente nueva. Su propósito es continuar, completar y depurar la actividad de detección de ataques de fuerza bruta con ELK iniciada en la sesión anterior.

Durante la clase, cada estudiante debe terminar de conectar los componentes del pipeline:

1. VM en Google Cloud Platform.
2. Reglas de firewall para acceder a Elasticsearch y Kibana.
3. Stack ELK ejecutándose con Docker Compose.
4. Índice de Elasticsearch para almacenar logs.
5. Agente Python para recolectar logs de `/var/log`.
6. Consultas REST para validar datos.
7. Kibana Discover para explorar eventos.
8. Dashboard para visualizar intentos fallidos, usuarios e IPs origen.

La meta es que el estudiante pueda explicar cómo un conjunto de logs crudos se transforma en evidencia útil para detectar actividad sospechosa.

2. Relación con la sesión anterior

En la sesión anterior se trabajó la actividad de detección de fuerza bruta con ELK. Esa actividad incluía:

- abrir puertos en GCP;
- levantar Elasticsearch y Kibana;
- crear el índice `gcp-vm-logs`;
- definir mappings;
- ejecutar un script de ingesta en Python;
- consultar datos con la API REST;
- crear un Data View en Kibana;
- analizar eventos de autenticación fallida;
- construir visualizaciones.

Esta clase práctica está destinada a terminar lo que no haya quedado funcionando, corregir errores y construir el reporte final con evidencias verificables.

3. Resultado esperado

Al finalizar la sesión, cada estudiante deberá tener:

1. Elasticsearch respondiendo por API REST.
2. Kibana accesible desde navegador.
3. El índice `gcp-vm-logs` creado.
4. Logs enviados desde la VM hacia Elasticsearch.
5. Eventos visibles en Kibana Discover.
6. Al menos una consulta KQL para detectar fallos de autenticación.
7. Al menos dos visualizaciones en Kibana.
8. Un dashboard básico de análisis.
9. Evidencias suficientes para el reporte.
10. Una reflexión técnica sobre riesgos, costos y límites de la solución.

4. Regla ética y de seguridad

Uso defensivo y autorizado

Esta práctica es defensiva y educativa. Solo se permite analizar logs de la infraestructura propia del estudiante o de recursos autorizados por el profesor.

No se debe intentar generar tráfico, escaneo, autenticaciones fallidas ni pruebas de fuerza bruta contra sistemas de terceros. Si se requiere generar eventos de autenticación fallida para tener datos de prueba, deben generarse únicamente contra la VM propia del estudiante y de forma controlada.

5. Arquitectura de referencia

La arquitectura mínima esperada es:

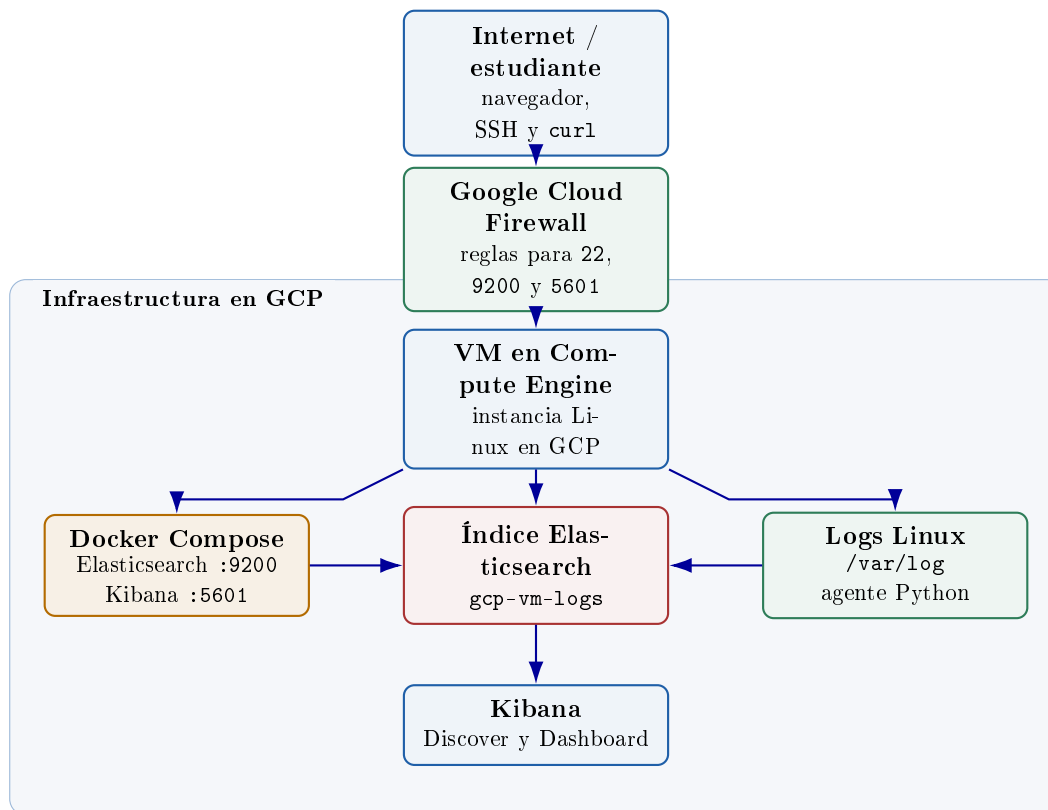


Figura: Flujo mínimo de recolección, ingesta, almacenamiento y análisis de logs.

6. Puertos utilizados

Puerto	Servicio	Uso
22	SSH	Administración remota de la VM.
9200	Elasticsearch	API REST e ingesta de documentos.
5601	Kibana	Interfaz web de análisis.

Los puertos 9200 y 5601 deben abrirse solo para la práctica y preferentemente restringirse a la IP pública del estudiante cuando sea posible.

7. Organización de la clase

7.1. Diagnóstico inicial del avance

Cada estudiante debe comenzar identificando en qué punto se encuentra:

- no tengo VM;
- tengo VM, pero no tengo Docker;
- Docker está instalado, pero ELK no levanta;
- Elasticsearch levanta, pero Kibana no responde;
- Kibana responde, pero no hay datos;
- hay datos, pero no se clasifican eventos de autenticación;
- hay datos y necesito construir visualizaciones;
- ya tengo dashboard y necesito documentar.

El profesor revisará avances a partir de esta lista y priorizará bloqueos.

7.2. Verificación de la VM

Entrar por SSH a la VM y ejecutar:

```
1 hostname
2 whoami
3 cat /etc/os-release
4 free -h
5 df -h
6 ip addr
```

Estas salidas ayudan a confirmar sistema operativo, usuario, memoria, disco y red.

7.3. Verificación de firewall en GCP

Revisar que existan reglas de entrada para los puertos 9200 y 5601, y que apliquen a la VM correcta. Si se usa etiqueta de red, confirmar que la VM tenga la etiqueta esperada:

```
1 gcloud compute instances describe NOMBRE_VM --zone ZONA_VM
```

Buscar la sección `tags`.

Ejemplo de reglas de firewall:

```
1 gcloud compute firewall-rules create allow-kibana-ingress \  
2   --allow=tcp:5601 \  
3   --direction=INGRESS \  
4   --priority=1000 \  
5   --network=default \  
6   --target-tags=elk-server  
7  
8 gcloud compute firewall-rules create allow-elasticsearch-rest \  
9   --allow=tcp:9200 \  
10  --direction=INGRESS \  
11  --priority=1000 \  
12  --network=default \  
13  --target-tags=elk-server
```

Asignar la etiqueta a la VM:

```
1 gcloud compute instances add-tags NOMBRE_VM \  
2   --tags=elk-server \  
3   --zone=ZONA_VM
```

Si la regla ya existe, no se debe duplicar. En ese caso se revisa la regla existente.

7.4. Verificación de Docker y ELK

Dentro de la VM:

```
1 docker --version  
2 docker compose version  
3 docker ps
```

Si el repositorio `docker-elk` ya existe:

```
1 cd ~/docker-elk
```

```
2 docker compose ps
3 docker compose logs --tail=80 elasticsearch
4 docker compose logs --tail=80 kibana
```

Si el stack no está arriba:

```
1 cd ~/docker-elk
2 docker compose up -d
```

Validar puertos:

```
1 ss -tulpn | grep -E "9200|5601"
```

Resultado esperado:

- Elasticsearch escuchando en 9200.
- Kibana escuchando en 5601.
- Contenedores en estado Up.

7.5. Validación de Elasticsearch

Desde la VM:

```
1 curl http://localhost:9200/_cluster/health?pretty
```

Desde la computadora local, usando la IP pública:

```
1 curl http://IP_PUBLICA_VM:9200/_cluster/health?pretty
```

El estado **yellow** es aceptable en un laboratorio de un solo nodo. Significa que los shards primarios funcionan, aunque no hay réplicas en otro nodo.

7.6. Validación de Kibana

Abrir en el navegador:

```
1 http://IP_PUBLICA_VM:5601
```

Si no abre:

- revisar firewall de GCP;
- revisar si el contenedor de Kibana está arriba;

- revisar logs de Kibana;
- confirmar que la URL usa `http://`;
- confirmar que la IP pública corresponde a la VM correcta.

8. Creación o revisión del índice

El índice esperado es:

```
1 gcp-vm-logs
```

Desde Kibana Dev Tools o usando `curl`, revisar:

```
1 curl http://localhost:9200/gcp-vm-logs?pretty
```

Si el índice no existe, crearlo con un mapping explícito:

```
1 curl -X PUT "http://localhost:9200/gcp-vm-logs" \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "mappings": {  
5       "properties": {  
6         "@timestamp": { "type": "date" },  
7         "log_file": { "type": "keyword" },  
8         "hostname": { "type": "keyword" },  
9         "host": { "type": "keyword" },  
10        "collector_host": { "type": "keyword" },  
11        "process": { "type": "keyword" },  
12        "event_type": { "type": "keyword" },  
13        "username": { "type": "keyword" },  
14        "source_ip": { "type": "ip" },  
15        "message": { "type": "text" },  
16        "raw_line": { "type": "text" },  
17        "raw_timestamp": { "type": "keyword" }  
18      }  
19    }  
20  }'
```

Si el índice ya existe con un mapping incorrecto, se debe documentar el problema antes de borrar. Para reiniciar el índice:

```
1 curl -X DELETE "http://localhost:9200/gcp-vm-logs"
```

Luego se vuelve a crear el índice con el mapping correcto.

9. Agente Python de ingesta

La carpeta de la actividad anterior contiene dos archivos de apoyo:

- `setup_env.sh`;
- `collect_varlog_to_elk.py`.

El objetivo del agente es leer archivos de `/var/log`, transformar líneas de log en documentos JSON y enviarlos a Elasticsearch.

9.1. Preparar entorno Python

En la VM:

```
1 sudo apt update
2 sudo apt install -y python3 python3-venv python3-pip file
```

Crear entorno:

```
1 python3 -m venv elk-logs-env
2 source elk-logs-env/bin/activate
3 pip install requests
```

Si se usa `setup_env.sh`:

```
1 chmod +x setup_env.sh
2 ./setup_env.sh
3 source elk-logs-env/bin/activate
```

9.2. Revisar el script antes de ejecutarlo

Antes de correr el agente, cada estudiante debe revisar:

- URL de Elasticsearch;
- usuario y contraseña si el stack tiene autenticación;
- índice de destino;
- límite de líneas por archivo;
- exclusión de archivos binarios;

- permisos de lectura sobre `/var/log`.

Ejemplo de endpoint si el script corre dentro de la misma VM:

```
1 ELASTIC_URL = "http://localhost:9200/gcp-vm-logs/_doc"
```

Ejemplo de endpoint si el script corre desde otra máquina:

```
1 ELASTIC_URL = "http://IP_PUBLICA_VM:9200/gcp-vm-logs/_doc"
```

No se deben publicar contraseñas ni tokens en el reporte.

9.3. Ejecutar el agente

```
1 source elk-logs-env/bin/activate
2 python3 collect_varlog_to_elk.py
```

Si aparecen errores de permisos en `/var/log`, se puede ejecutar con `sudo`, pero documentando la razón:

```
1 sudo elk-logs-env/bin/python collect_varlog_to_elk.py
```

Resultado esperado:

- lista de archivos detectados;
- mensajes de procesamiento;
- conteo final de documentos enviados;
- documentos visibles en Elasticsearch.

10. Generación controlada de eventos de autenticación fallida

Si la VM no tiene suficientes eventos de autenticación fallida, se pueden generar intentos fallidos de forma controlada contra la VM propia.

Desde la computadora del estudiante:

```
1 ssh usuario_inexistente@IP_PUBLICA_VM
```

Ingresar una contraseña incorrecta y cancelar después de pocos intentos.

Reglas:

- hacerlo solo contra la VM propia;
- no automatizar diccionarios;

- no ejecutar ataques contra terceros;
- no saturar la VM;
- documentar que se generaron eventos controlados de prueba.

Después, volver a ejecutar el agente de ingesta para enviar los nuevos logs a Elasticsearch.

11. Consultas REST de validación

11.1. Conteo total de documentos

```
1 curl "http://localhost:9200/gcp-vm-logs/_count?pretty"
```

11.2. Buscar eventos relacionados con autenticación fallida

Si el script clasifica `event_type`:

```
1 curl -X GET "http://localhost:9200/gcp-vm-logs/_search?pretty" \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "query": {  
5       "term": {  
6         "event_type": "authentication_failure"  
7       }  
8     }  
9   }'
```

Si todavía no existe `event_type`, buscar en el mensaje:

```
1 curl -X GET "http://localhost:9200/gcp-vm-logs/_search?pretty" \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "query": {  
5       "match": {  
6         "message": "Failed password"  
7       }  
8     }  
9   }'
```

11.3. Top de IPs origen

```
1 curl -X GET "http://localhost:9200/gcp-vm-logs/_search?pretty" \  
2 -H "Content-Type: application/json" \  
3 -d '{  
4     "size": 0,  
5     "aggs": {  
6         "top_source_ip": {  
7             "terms": {  
8                 "field": "source_ip",  
9                 "size": 10  
10            }  
11        }  
12    }  
13 }'
```

11.4. Top de usuarios

```
1 curl -X GET "http://localhost:9200/gcp-vm-logs/_search?pretty" \  
2 -H "Content-Type: application/json" \  
3 -d '{  
4     "size": 0,  
5     "aggs": {  
6         "top_usernames": {  
7             "terms": {  
8                 "field": "username",  
9                 "size": 10  
10            }  
11        }  
12    }  
13 }'
```

12. Trabajo en Kibana

12.1. Crear Data View

En Kibana:

1. Abrir http://IP_PUBLICA_VM:5601.

2. Ir a *Stack Management*.
3. Entrar a *Data Views*.
4. Crear un Data View con el patrón:

```
1 gcp-vm-logs*
```

5. Seleccionar `@timestamp` como campo de tiempo.
6. Guardar.

12.2. Explorar en Discover

Ir a *Analytics* → *Discover*.
Consultas KQL sugeridas:

```
1 event_type : "authentication_failure"
```

```
1 message : "Failed password"
```

```
1 message : "Invalid user"
```

```
1 source_ip : *
```

```
1 username : *
```

Ajustar el rango temporal a:

- Last 15 minutes;
- Last 1 hour;
- Last 24 hours;
- rango personalizado según la hora de la práctica.

12.3. Visualizaciones mínimas

El dashboard debe incluir al menos:

1. Conteo total de eventos.

2. Conteo de eventos de autenticación fallida.
3. Top de IPs origen.
4. Top de usuarios intentados.
5. Histograma temporal de eventos.
6. Tabla de eventos recientes con `@timestamp`, `source_ip`, `username`, `process` y `message`.

13. Guía de troubleshooting

13.1. Elasticsearch no responde

Revisar:

```
1 docker ps
2 docker compose logs --tail=80 elasticsearch
3 curl http://localhost:9200/_cluster/health?pretty
4 free -h
```

Posibles causas:

- contenedor caído;
- falta de memoria;
- puerto no expuesto;
- Docker no está ejecutando;
- Elasticsearch aún está iniciando.

13.2. Kibana no abre

Revisar:

```
1 docker ps
2 docker compose logs --tail=80 kibana
3 ss -tulpn | grep 5601
```

Posibles causas:

- Kibana aún está arrancando;
- firewall no permite 5601;
- IP pública incorrecta;

- contenedor caído;
- Elasticsearch no está listo.

13.3. El agente Python no envía documentos

Revisar:

```
1 python3 --version
2 source elk-logs-env/bin/activate
3 python3 -c "import requests; print('requests ok')"
4 curl http://localhost:9200/gcp-vm-logs/_count?pretty
```

Posibles causas:

- endpoint incorrecto;
- autenticación incorrecta;
- falta `requests`;
- permisos insuficientes para leer `/var/log`;
- índice no existe;
- mapping incompatible.

13.4. No aparecen eventos de fuerza bruta

Revisar:

- si existen líneas de autenticación fallida en `/var/log/auth.log`;
- si el sistema usa otro archivo de logs;
- si el parser reconoce los patrones;
- si `event_type` se está generando correctamente;
- si Kibana está filtrando un rango temporal incorrecto.

Buscar en la VM:

```
1 sudo grep -Ei "failed password|invalid user|authentication failure" /var/log/auth.log
```

Si no existe `/var/log/auth.log`, revisar:

```
1 ls -lah /var/log
2 journalctl -u ssh --no-pager | tail -50
```

13.5. Error de mapping

Si Elasticsearch rechaza documentos con `document_parsing_exception`, revisar:

- tipo del campo `source_ip`;
- campos enviados por Python;
- mapping del índice;
- documentos previos que hayan creado mapping dinámico incorrecto.

Ver mapping:

```
1 curl http://localhost:9200/gcp-vm-logs/_mapping?pretty
```

14. Evidencias mínimas para el reporte

El reporte debe incluir capturas o salidas de terminal de:

1. VM activa en GCP.
2. Reglas de firewall para 9200 y 5601.
3. Contenedores de ELK ejecutándose.
4. Respuesta de `/_cluster/health`.
5. Índice `gcp-vm-logs` creado.
6. Mapping del índice.
7. Ejecución del agente Python.
8. Conteo de documentos en Elasticsearch.
9. Consulta REST de eventos de autenticación fallida.
10. Kibana Data View creado.
11. Discover mostrando eventos.
12. Consulta KQL usada.
13. Dashboard con visualizaciones.
14. Problemas encontrados y soluciones.
15. Costos observados o estimados.
16. Medidas de limpieza o cierre de recursos.

15. Bitácora de trabajo

Durante la clase, cada estudiante debe mantener una bitácora mínima:

Hora	Acción realizada	Resultado	Problema encontrado	Solución aplicada

Esta bitácora debe alimentar el reporte final.

16. Preguntas de análisis para el reporte

Responder:

1. ¿Qué diferencia hay entre logs crudos y eventos estructurados?
2. ¿Por qué es útil transformar logs en documentos JSON?
3. ¿Qué función cumple Elasticsearch en esta práctica?
4. ¿Qué función cumple Kibana?
5. ¿Qué evidencia permite afirmar que hubo intentos de autenticación fallida?
6. ¿Qué usuario fue intentado con mayor frecuencia?
7. ¿Qué IP origen generó más eventos sospechosos?
8. ¿El patrón parece ataque dirigido o escaneo automatizado? Justificar.
9. ¿Qué limitaciones tiene el parser usado?
10. ¿Qué riesgos implica exponer SSH, Elasticsearch y Kibana a Internet?
11. ¿Qué regla de firewall o medida de hardening propondrías para reducir riesgo?
12. ¿Qué costos o consumo de recursos observaste en la VM?

17. Estructura sugerida del reporte

El PDF debe seguir esta estructura:

1. Portada.
2. Objetivo de la práctica.

3. Arquitectura implementada.
4. Configuración de GCP y firewall.
5. Despliegue de ELK.
6. Creación del índice y mapping.
7. Agente Python de ingesta.
8. Consultas REST de validación.
9. Exploración en Kibana Discover.
10. Dashboard creado.
11. Análisis de eventos de fuerza bruta.
12. Problemas encontrados y soluciones.
13. Riesgos y recomendaciones de seguridad.
14. Costos observados.
15. Conclusiones.
16. Evidencias.

18. Criterios de evaluación

Criterio	Ponderación
Stack ELK funcional	20 %
Ingesta correcta de logs	20 %
Consultas REST y validación de datos	15 %
Uso de Kibana Discover y Dashboard	20 %
Análisis de fuerza bruta	15 %
Reporte, evidencias y reflexión técnica	10 %

19. Cierre de la sesión

Antes de terminar, cada estudiante debe confirmar:

- que tiene capturas suficientes;
- que el índice contiene documentos;
- que puede consultar eventos en Discover;

- que guardó o documentó su dashboard;
- que anotó problemas y soluciones;
- que no publicó credenciales;
- que revisó costos;
- que apagó o protegió recursos si ya no los usará.

La práctica se considera completa cuando el estudiante puede demostrar el flujo completo: logs de Linux, ingesta con Python, almacenamiento en Elasticsearch, consulta REST, exploración en Kibana y análisis razonado de intentos de fuerza bruta.